My Source Code:

```
import java.time.LocalDate;

import java.util.ArrayList;

import java.util.concurrent.atomic.AtomicInteger;


public class Account {

protected String fullName;

protected static String identifier; //this is the first 2 digits that identify the type of account(01-Chequing, 02-Savings, 03-Investment).

protected int accountNo;

protected int clientID;

protected double balance;

protected double interestRate;

protected LocalDate dateOpened;

private static AtomicInteger nextAccountNumber = new AtomicInteger(1);

protected ArrayList<Transaction> transactions = new ArrayList<>();


public Account(String fullName, int clientID, LocalDate dateOpened) {

	super();

	this.fullName = fullName;

	this.identifier = identifier;

	this.accountNo = nextAccountNumber.getAndIncrement();

	this.clientID = clientID;

	this.balance = 0;

	this.interestRate = interestRate; //do i keep it???

	this.dateOpened = dateOpened;

}
```

```
// GETTERS AND SETTERS


public String getFullName() {
	return fullName;
}

public void setFullName(String fullName) {
	this.fullName = fullName;
}

public String getAccountNo() {
	return identifier + String.format("%04d", accountNo);
}

public void setAccountNo(int accountNo) {
	this.accountNo = accountNo;
}

public int getClientID() {
	return clientID;
}

public void setClientID(int clientID) {
	this.clientID = clientID;
}

public double getBalance() {
	return balance;
```

```
}

public void setBalance(double balance) {
	this.balance = balance;
}

public double getInterestRate() {
	return interestRate;
}

public void setInterestRate(double interestRate) {
	this.interestRate = interestRate;
}

public LocalDate getDateOpened() {
	return dateOpened;
}

public void setDateOpened(LocalDate dateOpened) {
	this.dateOpened = dateOpened;
}

//ACCOUNT MANAGEMENT

protected void deposit(int accountNo, double amount) {
	if(accountNo == this.accountNo) {
		balance = balance + amount;
		transactions.add(new Transaction("DEPOSIT", amount, "Deposit to account."));
		System.out.println("Transaction executed successfully.");
```

```
        }
        else {
                System.out.println("Invalid account number. This account number does not exist in
our database.");
        }
}

protected void deposit(double amount) {
        balance = balance + amount;
        transactions.add(new Transaction("DEPOSIT", amount, "Deposit to account."));
        System.out.println("Transaction executed successfully.");
}

protected void withdraw(int accountNo, double amount) throws InsufficientFundsException{
        if(accountNo == this.accountNo) {
                if(balance > amount) {
                        balance = balance - amount;
                        transactions.add(new Transaction("WITHDRAWAL", amount, "Withdrawal
from account."));
                        System.out.println("Transaction executed successfully.");
                }
                else {
                        throw new InsufficientFundsException("Insufficient funds. Withdrawal
cannot go through.");
                }
        }
        else {
                System.out.println("Invalid account number. This account number does not exist in
our database.");
        }
```

```
}



@Override
public String toString() {
	return "Client Name: " + fullName + "\nClientID: " + clientID + "\nBalance: " + balance +
"\nDate Opened: " + dateOpened;
}

protected void printTransactionHistory() {
	for(Transaction tr : transactions) {
		System.out.println(tr.toString());
	}
}

public Object getLastFeeDate() {
	return null;
}

public void setLastFeeDate(Object lastFeeDate) {
}
```

```
}

import java.time.LocalDate;

public class Chequing extends Account{

	public Chequing(String fullName,  int clientID, LocalDate dateOpened) {
		super(fullName, clientID, dateOpened);
		identifier = "01";

	}
//
	protected void deposit(int accountNo, double amount) {
		super.deposit(accountNo, amount);
	}
	protected void deposit(double amount) {
		super.deposit(amount);
	}

	protected void withdraw(int accountNo, double amount) throws
InsufficientFundsException{
		super.withdraw(accountNo, amount);
	}

}

public class InvestmentLockException extends Exception {
```

```
	public InvestmentLockException(String message) {
		super(message);
	}

}
import java.time.LocalDate;
import java.util.ArrayList;
import java.time.temporal.ChronoUnit;

public abstract class Client implements Maintainable{
	protected String type;
	protected String name;
	protected int clientID;
	protected int SSN;
	protected String password;
	protected String DOB;
	protected String phoneNo;
	protected String email;
	protected String address;
	private double monthlyFee = 10.0;
	protected LocalDate lastFeeDate;

	protected ArrayList<Account> clientAcc = new ArrayList<>();

	public Client(String name, int clientID, int sSN, String password, String dOB, String
phoneNo, String email,
			String address) {
		this.type = type;
```

```
        this.name = name;
        this.clientID = clientID;
        SSN = sSN;
        this.password = password;
        DOB = dOB;
        this.phoneNo = phoneNo;
        this.email = email;
        this.address = address;
}

//GETTERS AND SETTERS

public String getName() {
        return name;
}

public String getType() {
        return type;
}

public void setType(String type) {
        this.type = type;
}

public void setName(String name) {
        this.name = name;
}
```

```
public int getClientID() {
	return clientID;
}


public void setClientID(int clientID) {
	this.clientID = clientID;
}


public int getSSN() {
	return SSN;
}

public void setSSN(int sSN) {
	SSN = sSN;
}

public String getPassword() {
	return password;
}

public void setPassword(String password) {
	this.password = password;
}

public String getDOB() {
	return DOB;
}
```

```
public void setDOB(String dOB) {
	DOB = dOB;
}

public String getPhoneNo() {
	return phoneNo;
}

public void setPhoneNo(String phoneNo) {
	this.phoneNo = phoneNo;
}

public String getEmail() {
	return email;
}

public void setEmail(String email) {
	this.email = email;
}

public String getAddress() {
	return address;
```

```
	}



	public void setAddress(String address) {
		this.address = address;
	}

	//CLIENT GESTION METHODS
	protected abstract boolean hasChequingAccount();

	protected abstract void openNewAccount(String accountType, int clientID) throws
MissingChequingAccountException;

	protected abstract void transfer(Account from, Account to, double amount);

	protected boolean login(String enterPassword) {
		if(enterPassword.equals(this.password)) {
			return true;
		}
		else {
			return false;
		}
	}



	@Override
	public String toString() {
		return "Client Name:" + name + "\nClientID: " + clientID + "\nSSN: " + SSN + "\nDOB:"
+ DOB + "\nContact: " + "\n->Phone Number: "
```

```
                                        + phoneNo + "\n->Email:" + email + "\nAddress: " + address +
"\nClient Accounts: " + clientAcc;
        }

        //MAINTAINABLE METHOD
        public void applyMonthlyFee() {
                LocalDate currentDate = LocalDate.now();
                for(Account acc: clientAcc) {
                        if(acc instanceof Chequing) {
                                if(lastFeeDate == null ||
lastFeeDate.plusMonths(1).isAfter(currentDate)) {
                                        acc.setBalance(acc.getBalance() - monthlyFee);
                                        lastFeeDate = currentDate;
                                }

                        }
                }
        }

}
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;

public class PremiumClient extends Client implements Maintainable{

        public PremiumClient(String name, int clientID, int sSN, String password, String dOB, String
phoneNo, String email,
                        String address) {
                super(name, clientID, sSN, password, dOB, phoneNo, email, address);
```

```
            super.type = type;
        }

        //CLIENT GESTION METHODS
        protected boolean hasChequingAccount() {
            for(Account acc : clientAcc) {
                if(acc instanceof Chequing) {
                    return true;
                }
            }
            return false;
        }

        protected void openNewAccount(String accountType, int clientID) throws
MissingChequingAccountException {
            if(hasChequingAccount() == true) {
                if(accountType.equalsIgnoreCase("Chequing")) {
                    Chequing c = new Chequing(name, clientID, LocalDate.now());
                }
                else if (accountType.equalsIgnoreCase("Savings")) {
                    Savings s = new Savings (name, clientID, LocalDate.now());
                }
                else if(accountType.equalsIgnoreCase("Investment")) {
                    Investment i = new Investment(name, clientID, LocalDate.now());


                }
            }
            else {
                try {
```

```
                    if(!hasChequingAccount()) {
                        throw new MissingChequingAccountException("You can't open an investment account without a chequing account.");
                    }
                }
                    catch(MissingChequingAccountException e){
                        System.out.println(e);
                    }
                }


                }

@Override
        protected void transfer(Account from, Account to, double amount) {
                try {
                    from.withdraw(from.accountNo, amount);
                    from.transactions.add(new Transaction("TRANSFER_OUT", amount, "Transfer from account."));
                    to.deposit(to.accountNo, amount);
                    to.transactions.add(new Transaction("TRANSFER_IN", amount, "Transfer into account."));
                }
                catch(InsufficientFundsException e) {
                    System.out.println(e);
                }
        }


        protected boolean login(String enterPassword) {
                return super.login(enterPassword);
        }
```

```
	public String toString() {
		return super.toString();
	}

	//MAINTAINABLE METHOD
@Override
	public void applyMonthlyFee() {
		super.applyMonthlyFee();
	}



}
import java.time.LocalDate;

public class VIPClient extends PremiumClient implements Maintainable{

	public VIPClient(String name, int clientID, int sSN, String password, String dOB, String phoneNo, String email,
			String address) {
		super(name, clientID, sSN, password, dOB, phoneNo, email, address);
		this.type = "VIP";
	}

	//CLIENT GESTION METHODS
	protected boolean hasChequingAccount() {
		return super.hasChequingAccount();
	}
```

```
    protected void openNewAccount(String accountType, int clientID) throws
MissingChequingAccountException {
        super.openNewAccount(accountType, clientID);
    }

    protected void transfer(Account from, Account to, double amount) {
        super.transfer(from, to, amount);

    }

    protected boolean login(String enterPassword) {
        return super.login(enterPassword);
    }

    private void addOnePercent() {
        for(Account acc : clientAcc){
            if(acc instanceof Savings || acc instanceof Investment) {
                acc.interestRate =+ 0.01;
            }
        }
    }

    public String toString() {
        return super.toString();
    }

    //MAINTAINABLE METHOD
    public void applyMonthlyFee() {
        System.out.println("No monthly fee. Your fees were waived.");
```

```
	}
}
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class CorporateClient extends PremiumClient implements Maintainable{
private double monthlyFee = 10.0;

	public CorporateClient(String name, int clientID, int sSN, String password, String dOB,
String phoneNo,
				String email, String address) {
		super(name, clientID, sSN, password, dOB, phoneNo, email, address);
		this.type = "Corporate";
	}

	//CLIENT GESTION METHODS
	protected boolean hasChequingAccount() {
		return super.hasChequingAccount();
	}

	protected void openNewAccount(String accountType, int clientID) throws
MissingChequingAccountException {
		super.openNewAccount(accountType, clientID);
		}

	protected void transfer(Account from, Account to, double amount) {
		super.transfer(from, to, amount);

	}
```

```
	protected boolean login(String enterPassword) {
		return super.login(enterPassword);
	}

	public String toString() {
		return super.toString();
	}

	//MAINTAINABLE METHOD
	public void applyMonthlyFee() {
		super.applyMonthlyFee();
	}
}
import java.time.LocalDate;

public class StandardClient extends Client implements Maintainable {

	public StandardClient(String name, int clientID, int sSN, String password, String dOB,
String phoneNo, String email,
			String address) {
		super(name, clientID, sSN, password, dOB, phoneNo, email, address);
		super.type = type;
	}

	//CLIENT GESTION METHODS
	protected boolean hasChequingAccount() {
		for(Account acc : clientAcc) {
			if(acc instanceof Chequing) {
				return true;
			}
```

```
            }
            return false;
        }

        protected void openNewAccount(String accountType, int clientID) throws
MissingChequingAccountException {
            if(hasChequingAccount() == true) {
                if(accountType.equalsIgnoreCase("Chequing")) {
                    Chequing c = new Chequing(name, clientID, LocalDate.now());
                }
                else if (accountType.equalsIgnoreCase("Savings")) {
                    Savings s = new Savings (name, clientID, LocalDate.now());
                }
                else if(accountType.equalsIgnoreCase("Investment")) {
                    Investment i = new Investment(name, clientID, LocalDate.now());

                }
            }
            else {
                try {
                if(!hasChequingAccount()) {
                    throw new MissingChequingAccountException("You can't open an
investment account without a chequing account.");
                }
            }
                catch(MissingChequingAccountException e){
                    System.out.println(e);
                }
            }
```

```
        }

    @Override
    protected void transfer(Account from, Account to, double amount) {
        try {
            from.withdraw(from.accountNo, amount);
            from.transactions.add(new Transaction("TRANSFER_OUT", amount,
"Transfer from account."));
            to.deposit(to.accountNo, amount);
            to.transactions.add(new Transaction("TRANSFER_IN", amount, "Transfer
into account."));
        }
        catch(InsufficientFundsException e) {
            System.out.println(e);
        }

    }

    protected boolean login(String enterPassword) {
        return super.login(enterPassword);
    }

    public String toString() {
        return super.toString();
    }

    //MAINTAINABLE METHOD
    public void applyMonthlyFee() {
        super.applyMonthlyFee();
    }
```

```
}
import java.time.LocalDate;


public class IndividualClient extends StandardClient implements Maintainable {
	double monthlyFee = 10.0;
	public IndividualClient(String name, int clientID, int sSN, String password, String dOB,
String phoneNo,
				String email, String address) {
		super(name, clientID, sSN, password, dOB, phoneNo, email, address);
		this.type = "Individual";
	}

	//CLIENT GESTION METHODS
	protected boolean hasChequingAccount() {
		return super.hasChequingAccount();
	}

	protected void openNewAccount(String accountType, int clientID) throws
MissingChequingAccountException {
		super.openNewAccount(accountType, clientID);
	}

	protected void transfer(Account from, Account to, double amount) {
		super.transfer(from, to, amount);


	}

	protected boolean login(String enterPassword) {
		return super.login(enterPassword);
```

```
	}

	public String toString() {
		return super.toString();
	}

	//MAINTAINABLE METHOD
	public void applyMonthlyFee() {
		super.applyMonthlyFee();
	}
}
import java.time.LocalDate;

public class StudentClient extends StandardClient implements Maintainable {

	public StudentClient(String name, int clientID, int sSN, String password, String dOB, String phoneNo, String email,
			String address) {
		super(name, clientID, sSN, password, dOB, phoneNo, email, address);
		this.type = "Student";
	}

	//ACCOUNT GESTION METHODS
	protected boolean hasChequingAccount() {
		return super.hasChequingAccount();
	}

	protected void openNewAccount(String accountType, int clientID) throws MissingChequingAccountException {
		super.openNewAccount(accountType, clientID);
```

```
        }

        protected void transfer(Account from, Account to, double amount) {
                super.transfer(from, to, amount);

        }

        protected boolean login(String enterPassword) {
                return super.login(enterPassword);
        }

        public String toString() {
                return super.toString();
        }

        //MAINTAINABLE METHOD
        public void applyMonthlyFee() {
                System.out.println("No monthly fee. Your fees were waived.");
        }

}
public class InvestmentLockException extends Exception {

        public InvestmentLockException(String message) {
                super(message);
        }

}
```

```
public class MissingChequingAccountException extends Exception {

public MissingChequingAccountException(String message) {
        super(message);
}
}
public class InsufficientFundsException extends Exception {

        public InsufficientFundsException(String message) {
                super(message);
        }

}
public interface Maintainable {

        public void applyMonthlyFee();
}
public interface InterestBearing {

        public void applyInterest();
}

import java.time.LocalDate;
import java.util.concurrent.atomic.AtomicInteger;

public class Transaction {

        private String transactionType;
        private double amount;
```

```
private String description;
private LocalDate transactionDate;
private int transactionID;
private static AtomicInteger nextTransactionID = new AtomicInteger(1);

public Transaction(String transactionType, double amount, String description) {
        super();
        this.transactionID = nextTransactionID.getAndIncrement();
        this.transactionType = transactionType;
        this.amount = amount;
        this.description = description;
        this.transactionDate = LocalDate.now();
}


public String getTransactionType() {
        return transactionType;
}


public void setTransactionType(String transactionType) {
        this.transactionType = transactionType;
}


public double getAmount() {
        return amount;
}
```

```
public void setAmount(double amount) {
        this.amount = amount;
}


public String getDescription() {
        return description;
}


public void setDescription(String description) {
        this.description = description;
}


public LocalDate getTransactionDate() {
        return transactionDate;
}


public void setTransactionDate(LocalDate transactionDate) {
        this.transactionDate = transactionDate;
}


public int getTransactionID() {
        return transactionID;
}
```

```
	public void setTransactionID(int transactionID) {
		this.transactionID = transactionID;
	}

	@Override
	public String toString() {
		return transactionID + " | " + transactionDate + " | " + transactionType + " | $" +
amount + " | " + description;
	}

}
import com.google.gson.TypeAdapter;
import com.google.gson.stream.JsonReader;
import com.google.gson.stream.JsonWriter;
import java.io.IOException;
import java.time.LocalDate;

public class LocalDateAdapter extends TypeAdapter<LocalDate>{

	@Override
	public LocalDate read(JsonReader in) throws IOException {
		if (in.peek() == com.google.gson.stream.JsonToken.NULL) {
	    return null;
	}
	return LocalDate.parse(in.nextString());
	}
```

```
        @Override
        public void write(JsonWriter out, LocalDate ld) throws IOException {
                if (ld == null) {
            out.nullValue();
          } else {
            out.value(ld.toString()); // "2026-05-02"
          }

        }

}
import java.util.ArrayList;
import java.util.List;
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;

import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.util.Arrays;
import com.google.gson.Gson;
import com.google.gson.reflect.TypeToken;

import java.io.FileReader;
import java.lang.reflect.Type;
import java.util.List;
```

```
public class BankAppData {

	static ArrayList<Account> accountData = new ArrayList<>();
	static ArrayList<Client> clientData = new ArrayList<>();



	public BankAppData(ArrayList<Account> accountData, ArrayList<Client> clientData) {
		super();
		this.accountData = accountData;
		this.clientData = clientData;
	}

	public static void main(String[] args) {
		BankAppData app = new BankAppData(new ArrayList<>(), new ArrayList<>());

		//Sample data
				//Clients
		clientData.add(new StudentClient("Joe Davis", 1234, 8120063, "JesuisJoe", "2007-09-12", "514-223-8898", "JD@gmail.com", "1468 Park Avenue"));
		clientData.add(new VIPClient("Maryam Lopez", 4548, 5557609, "PumpkingIsG00d", "1991-12-12", "514-334-0021", "MaryamLpz@gmail.com", "112 Rainbow Road"));
		clientData.add(new CorporateClient("Laurence&Smith", 7752, 1231212, "L!veLaughL0v3", "1980-05-25", "514-825-4833", "Laurence&Smith@gmail.com", "8873 Clifton Hill"));
			//Accounts
		accountData.add(new Chequing("Joe Davis", 1234, LocalDate.now()));
		accountData.add(new Savings("Joe Davis", 1234, LocalDate.now()));
		accountData.add(new Chequing("Maryam Lopez", 4548, LocalDate.now()));
```

```
        accountData.add(new Chequing("Laurence&Smith", 7752, LocalDate.now()));


        //Write to GSON
        Gson gson = new GsonBuilder().registerTypeAdapter(LocalDate.class, new
LocalDateAdapter()).setPrettyPrinting().create();
        String filePath = "BankApplicationData.json";
        BankAppData bankData = new BankAppData(accountData, clientData);
            try(FileWriter fw = new FileWriter(filePath)) {
                gson.toJson(bankData, fw);
            }
            catch(IOException e) {
                System.out.println(e.getMessage());
            }
        //Read from GSON

            try(FileReader fr = new FileReader(filePath)){
                BankAppData bData = gson.fromJson(fr, BankAppData.class);
                ArrayList<Account> aData = new ArrayList<>(bData.accountData);
                for(Account acc : aData) {
                    Account rebuilt;

                    switch(acc.identifier) {
                      case "01":
                            rebuilt = new Chequing(acc.getAccountNo(),
acc.getClientID(), acc.getDateOpened());
                        break;

                      case "02":
                            rebuilt = new Savings(acc.getAccountNo(),
acc.getClientID(), acc.getDateOpened());
```

```
                    break;


                case "03":
                    rebuilt = new Investment(acc.getAccountNo(),
acc.getClientID(), acc.getDateOpened());
                    rebuilt.setLastFeeDate(acc.getLastFeeDate());
                    break;
                }
            }
            ArrayList<Client> cData = new ArrayList<>(bData.clientData);
            for(Client cl : cData) {
                Client rebuilt;
                    switch(cl.type) {
                    case "Individual":
                        System.out.println("Individual Client" + "\n");
                        rebuilt = new IndividualClient(cl.getName(),
cl.getClientID(), cl.getSSN(), cl.getPassword(), cl.getDOB(), cl.getPhoneNo(), cl.getEmail(),
cl.getAddress());
                        break;
                    case "Student":
                        System.out.println("Student Client" + "\n");
                        rebuilt = new StudentClient(cl.getName(),
cl.getClientID(), cl.getSSN(), cl.getPassword(), cl.getDOB(), cl.getPhoneNo(), cl.getEmail(),
cl.getAddress());
                        break;
                    case "Corporate":
                        System.out.println("Corporate Client" + "\n");
                        rebuilt = new CorporateClient(cl.getName(),
cl.getClientID(), cl.getSSN(), cl.getPassword(), cl.getDOB(), cl.getPhoneNo(), cl.getEmail(),
cl.getAddress());
                        break;
```

```
                                        case "VIP":
                                                System.out.println("VIP Client" + "\n");
                                                rebuilt = new VIPClient(cl.getName(), cl.getClientID(),
cl.getSSN(), cl.getPassword(), cl.getDOB(), cl.getPhoneNo(), cl.getEmail(), cl.getAddress());
                                                break;
                                        }
                                }
                        }
                        catch (IOException e) {
                          System.out.println(e.getMessage());
                        }


        }
}
```

The Snapshot Result.: